

Software Requirements Specification
for
Task Management System

Prepared by: Makmot Kabira Johnson, Armand Shematsi, Chemutai Marvis, Bbale Daniel

Bwanika

Department of Networks

11th June, 2026

Table of Contents

List of Figures	3
List of Tables	4
1. Introduction	6
1.1 Purpose	6
1.2 Intended Audience and Reading Suggestions	6
1.3 Product Scope	7
2. Overall Description	8
2.1 Product Perspective	8
2.2 Product Functions	11
2.3 User Classes and Characteristics	12
2.4 Operating Environment	13
2.5 Design and Implementation Constraints	14
2.6 User Documentation	15
2.7 Assumptions and Dependencies	15
3. External Interface Requirements	16
3.1 User Interfaces	16
3.2 Hardware Interfaces	18
3.3 Software Interfaces	19
3.4 Communications Interfaces	19
4. System Features	20
4.1 Task Management	20
4.2 Performance Monitoring	25
4.3 Machine Learning Engagement Monitoring	31
5. Non-Functional Requirements	33
5.1 Performance Requirements	33
5.2 Security Requirements	34
5.3 Reliability and Availability	34
5.4 Usability Requirements	34

5.5 Maintainability	35
5.6 Data Privacy and Compliance	35
6. References	37
Appendix A: Glossary	37
Appendix B: Use Case Summary	39

List of Figures

<i>Figure 1: Task Management System – System Architecture</i>	9
<i>Figure 2: Use Case Diagram of the Task Management System</i>	12
<i>Figure 3: Entity-Relationship Diagram – Task Management System</i>	36

List of Tables

<i>Table 1: Use Case UC1 — Task Management: Stimulus/Response Sequence</i>	18
<i>Table 2: SRS Detail — Task Management</i>	21
<i>Table 3: Use Case UC7 – View Analytics Dashboard: Stimulus/Response Sequence</i>	25
<i>Table 4: SRS Detail – Performance Monitoring</i>	27
<i>Table 5: Use Case UC8 – Generate Engagement Score and Risk Alerts: Stimulus/Response Sequence</i>	29
<i>Table 6: SRS Detail – Machine Learning Engagement Monitoring</i>	30
<i>Table 7: Glossary of Terms</i>	35
<i>Table 8: Use Case Summary</i>	37

1. Introduction

1.1 Purpose

This document defines the Software Requirements Specification (SRS) for the **Task Management System**. The system is designed to digitize and streamline the supervision of members within an organization by enabling supervisors to assign tasks, monitor daily and weekly progress, collect reports and submissions, aggregate performance metrics and leverage machine learning to track member engagement. It also provides supervisors with a structured mechanism to leave feedback and comments on member-submitted reports.

This document covers the full scope of the application, its functional requirements, non-functional requirements, external interface requirements and system constraints.

1.2 Intended Audience and Reading Suggestions

This document is intended to be read by the following parties:

- i. Project Managers and Development Team: Must become thoroughly familiar with all functional and non-functional requirements, particularly Sections 4 and 5, to ensure accurate implementation.
- ii. Testers and QA Team: Should focus on Section 4 (System Features) and Section 5 (Non-Functional Requirements) to develop meaningful and comprehensive test cases.
- iii. Marketing and Product Staff: Should review Section 1.4 (Product Scope) and Section 3 (External Interface Requirements) to understand product features for communication purposes.
- iv. Organizational Stakeholders: Should review Sections 1 and 2 to understand the goals, benefits, and constraints of the system.

- v. Documentation Writers: Should review all sections to produce accurate user manuals and in-app guides.

It is recommended that all readers start with Sections 1 and 2 before moving into the more technical sections.

1.3 Product Scope

The **Task Management System** is a web-based platform designed to replace manual, spreadsheet-driven member supervision workflows. It brings together supervisors and members in a single, organized digital environment where tasks can be assigned, progress tracked, reports submitted and reviewed, and performance trends automatically analyzed.

1.3.1 Goals

- i. To provide supervisors with a centralized digital workspace to manage all member-related activities.
- ii. To give members a clear, structured view of their assignments, deadlines, and expectations.
- iii. To automate performance tracking and reduce the administrative burden on supervisors.
- iv. To identify disengaged or struggling members early through machine learning, enabling timely intervention.
- v. To maintain a complete and auditable record of member activities, report submissions, and supervisor feedback.

1.3.2 Objectives

Main Objective

To build a reliable, scalable, and user-friendly member management platform that uses automation and machine learning to improve member supervision outcomes.

Other Objectives:

- i. To collect and analyze requirements for the Member Management System.

- ii. To design a relational data model and system architecture suitable for multi-user, role-based access.
- iii. To implement the system based on the approved design and requirements.
- iv. To test the system thoroughly with both supervisors and members before deployment.

1.3.3 Benefits

- i. Supervisors gain a real-time overview of each member's workload and progress without manual follow-ups.
- ii. Members receive structured feedback through annotated report comments, improving their professional development.
- iii. Management gains access to aggregated analytics dashboards showing cohort-level performance trends.
- iv. The machine learning engine flags at-risk members automatically, allowing proactive support.
- v. All member records, task histories, and submitted reports are stored securely in one place, reducing the risk of data loss.

2. Overall Description

2.1 Product Perspective

The **Task Management System** is a modern, standalone web application designed to replace the manual, spreadsheet-based tracking of members. The system is organized into three main architectural components that work together:

- i. **The Web Dashboard:** The primary interface that users log into to fill out forms, view charts, manage tasks, submit reports, and interact with the system's features. It is accessible via standard web browsers on any device.
- ii. **The Database (Filing Cabinet):** A structured, secure relational SQL database that stores all member profiles, task assignments, report submissions, performance logs, and activity history. It ensures referential integrity so that no record is accidentally lost, overwritten, or disassociated.

- iii. The Machine Learning Engine (Smart Assistant): A background processing module that continuously reads member activity logs to identify behavioral patterns. If a member's login frequency drops significantly or task completion falls below a defined threshold, the engine automatically flags that member as "at risk" and surfaces the alert on the supervisor's dashboard.

System Architecture

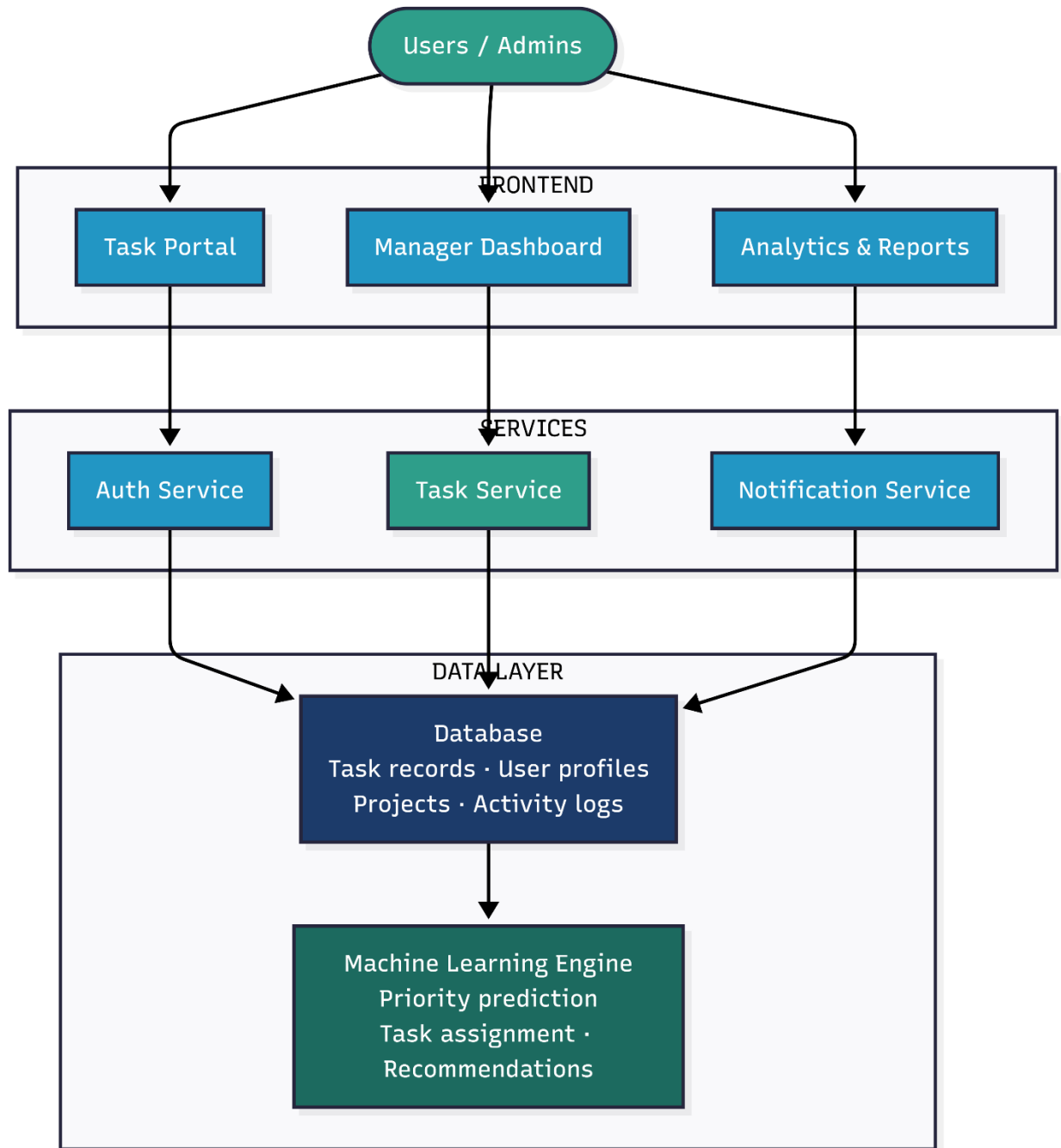


Figure 1: Task Management System – System Architecture

2.1.1 Site Adaptation Requirements

The system must be deployable on standard web hosting infrastructure or cloud platforms (e.g., AWS, Digital Ocean or a local on-premise server). It must support multi-user configurations where multiple supervisors and members can operate simultaneously without data crossover. The

interface must be fully responsive and usable on desktop browsers, laptops, tablets, and smartphones.

2.2 Product Functions

The Task Management System is built around four core functional areas:

- i. **Task Assignment and distribution:** Supervisors are provided with a digital dashboard to create tasks, divide these tasks into subtasks and assign them to specific members, and set distinct deadlines for these tasks. This eliminates reliance on scattered email threads or informal communication channels.
- ii. **Progress Tracking and work submission:** Members can update their task status (e.g., To-Do, In Progress, Completed) at any time. Supervisors can view a real-time board showing the progress of every member in their group, filtered by day or week. To prevent members from falsely marking status as completed or under review, the system strictly requires proof of work by directly uploading the weekly report file onto the platform.
- iii. **ML-Driven Performance Analytics and Monitoring:** The system quietly gathers raw operational statistics as they happen such as task completion dates, missed deadlines and system login times. The ML engine processes these metrics to compile clean summary totals (total tasks completed vs. late submissions) and calculates an overall engagement score for each member. The system then takes these scores and generates clear charts and automated alert lists on the supervisor's control panel, allowing them to instantly identify high-performing individuals and flag members who are falling behind and need immediate support.
- iv. **Supervisor Comment and Feedback System:** Supervisors can leave annotated comments directly on submitted member reports. Members receive notifications when new comments are added and can view all feedback from within their dashboard.

2.3 User Classes and Characteristics

The platform enforces role-based access control across three distinct user classes:

2.3.1 System Administrator

The System Administrator holds the highest privilege level. Their responsibilities include provisioning supervisor accounts, onboarding new members into the system, managing user roles, configuring system-level settings, and ensuring the health and uptime of the application. Administrators are expected to have technical proficiency in web-based systems.

2.3.2 Supervisor

Supervisors are the primary power users of the platform. They use the system daily to create and assign tasks, review submitted member reports, leave comments and feedback on reports, monitor progress through visual dashboards, and respond to automated ML-generated engagement alerts. Supervisors are expected to be proficient with standard web applications but do not require technical training.

2.3.3 Member

Members interact with the system through a simplified personal workspace. They can view their assigned tasks, update task statuses, submit daily logs and weekly reports, and read supervisor feedback on their submissions. Members cannot access other members' profiles, data, or managerial analytics views. Basic web browsing skills are sufficient for a member to use the system.

Use Case Diagram

USE CASE DIAGRAM OF THE TASK MANAGEMENT SYSTEM

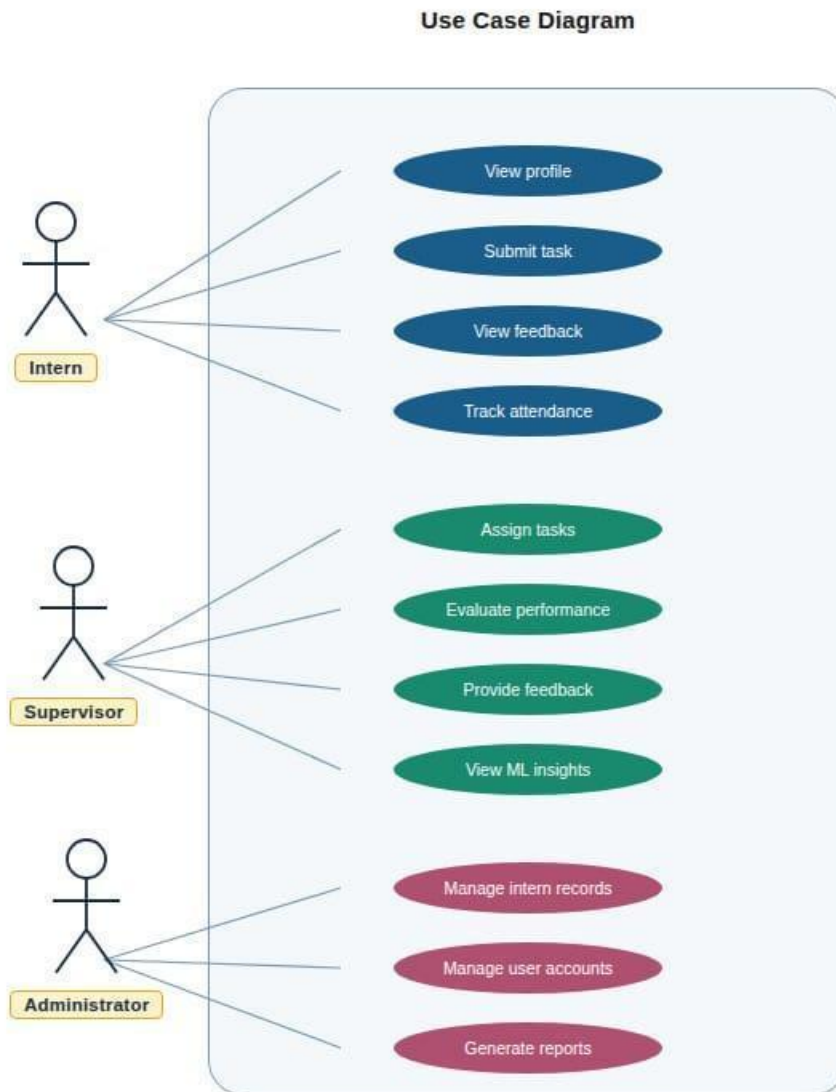


Figure 2: Use Case Diagram of the Task Management System

2.4 Operating Environment

The Member Management System will operate within a distributed multi-tier client-server environment and must function correctly under the following configuration:

- i. Server Side: A Linux-based web server running Python (Django or Flask) with a relational MySQL database and a background ML processing worker (Python with scikit-learn and pandas).
- ii. Client Side: Any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) on desktops, laptops, tablets, or smartphones.
- iii. Network: Requires a stable internet connection for real-time communication between client and server. A minimum bandwidth of 5 Mbps is recommended for a smooth experience.
- iv. Operating Systems: Compatible with Windows, macOS, Linux, Android, and iOS on the client side.

2.5 Design and Implementation Constraints

- i. Database Constraints: The system must use a relational SQL database (MySQL). All data entities, members, tasks, reports, and metrics are tightly interdependent, and a relational schema ensures referential integrity. A task record, for example, must always be linked to a valid member in the system.
- ii. ML Performance Boundary: The machine learning engine must run as an isolated background process, completely separate from the main web application codebase. Heavy computational operations (e.g., model training, engagement score recalculation) must not block or slow down the user-facing dashboard. Asynchronous task queues (e.g., Celery with Redis) should be used for this purpose.
- iii. Hardware Limitations: The platform must be lightweight and performant on standard office hardware. Client devices are assumed to have at minimum a dual-core processor, 4 GB RAM, and a modern browser. The server should be provisioned with at least 4 vCPUs and 8 GB RAM to handle concurrent users comfortably.
- iv. Data Privacy and Regulatory Compliance: Because the system stores personally identifiable information (names, email addresses, phone numbers, work performance data), it must comply strictly with Uganda's Data Protection and Privacy Act. This

includes implementing access controls, data encryption at rest and in transit (HTTPS/TLS), and providing mechanisms for data deletion upon request.

- v. Role Boundary Enforcement: The system must enforce strict privilege boundaries at the API level, not just the UI level. An member must never be able to access another member's data or any supervisor-level analytics, regardless of how they access the system.

2.6 User Documentation

The following documentation will be delivered alongside the software:

- i. In-App Quick Guides:
 - a. For Members: A simple visual guide (accessible from the dashboard) explaining how to view assigned tasks, update task statuses, and upload weekly reports.
 - b. For Supervisors: A contextual walkthrough explaining how to create and assign tasks, interpret dashboard analytics, read ML-generated engagement risk alerts, and leave comments on member reports.
- ii. System Administration Manual: A comprehensive guide covering user provisioning, role management, database setup, and system configuration for technical administrators.
- iii. Deployment and Setup Guide: A step-by-step technical manual explaining how to install the application, configure the SQL database, set up the ML background worker, and deploy to a hosting environment.

2.7 Assumptions and Dependencies

2.7.1 Assumptions

- i. All supervisors and members using the platform are assumed to have baseline familiarity with web browsers and standard online forms; no specialized technical training will be required for day-to-day use.

- ii. The organization implementing the system will provide and maintain adequate computing hardware and a stable internet connection for all users.
- iii. Supervisors are assumed to have sufficient familiarity with their members' work to provide meaningful written feedback through the comment system.
- iv. Member data entered into the system is assumed to be accurate and up-to-date; the system does not independently verify personal information beyond email confirmation.

2.7.2 Dependencies

- i. Internet Connectivity: The system depends on a stable and continuous internet connection. Without it, members cannot submit reports, and supervisors cannot monitor dashboards in real time.
- ii. Database Availability: The entire system depends on the continuous health of the MySQL database server. If the database becomes unavailable, all user profiles, tasks, reports, and analytics records will be inaccessible until it is restored.
- iii. Python ML Libraries: The engagement monitoring engine depends on the stability and availability of Python scientific libraries, specifically pandas for data processing and scikit-learn for behavioral modeling. Updates to these libraries must be tested before deployment.
- iv. Email Service Provider: Features such as account verification, supervisor notifications, and member report feedback alerts depend on a functioning email delivery service (e.g., SMTP via SendGrid or a similar provider).

3. External Interface Requirements

3.1 User Interfaces

The Member Management System will have the following primary interface screens:

- i. Login / Registration Page: The entry point for all users. Existing users log in using their registered email and password. New members are provisioned by the system administrator, who then sends them a registration link via email. The interface includes fields for email address and password, a "Forgot Password" link, and clear error messaging for incorrect credentials.

- ii. Member Dashboard: Upon login, an member is presented with their personal dashboard displaying:
 - A list of all currently assigned tasks, organized by status: To-Do, In Progress, and Completed.
 - A report submission panel where the member can type a daily log or upload a weekly progress report (PDF or DOCX).
 - A notifications panel showing new supervisor comments on recent submissions.
 - A simple summary card showing their current engagement score and task completion rate.
- iii. Supervisor Dashboard: Upon login, a supervisor sees:
 - An overview panel showing all members under their supervision with color-coded status indicators (green = on track, amber = moderate risk, red = high risk).
 - A task management panel for creating, editing, assigning, and tracking tasks.
 - A report review panel listing all recent member submissions, with the ability to open each report and leave inline comments.
 - An analytics section featuring visual charts generated from aggregated performance data.
 - An ML alert panel listing members currently flagged as disengaged.
- iv. Report Review and Comment Interface: When a supervisor opens an member's submitted report, they see the full report text or an embedded document viewer. A comment input panel appears on the right-hand side (or below on mobile), allowing the supervisor to type feedback and submit it. All existing comments on that report are listed in chronological order. The member receives a notification when a new comment is posted.
- v. System Administrator Panel: A back-end management interface for creating and managing user accounts, assigning roles, configuring system settings, and viewing system health metrics.

All interfaces must be fully responsive for mobile and tablet viewports. The UI should follow accessible design principles (WCAG 2.1 AA compliance is recommended).

3.2 Hardware Interfaces

The Member Management System is a web-based platform and therefore has no direct hardware interface dependencies. It is compatible with any device capable of running a modern web browser. The recommended minimum client hardware specifications are:

- Processor: Dual-core, 1.6 GHz or higher
- RAM: 4 GB or more
- Display: Any screen with a minimum resolution of 1024×768
- Network: Broadband internet connection (minimum 5 Mbps)

The recommended minimum server specifications are:

- Processor: 4 vCPUs (2.4 GHz or higher)
- RAM: 8 GB
- Storage: 80 GB SSD (scalable based on report and media storage needs)
- Operating System: Ubuntu Server 22.04 LTS or equivalent

3.3 Software Interfaces

The system will interface with the following external software components:

- i. MySQL Database Management System: All application data, user profiles, task records, report submissions, performance metrics and activity logs will be stored in a relational MySQL database. The system will use an ORM (e.g., Django ORM) to interact with the database.
- ii. Python ML Libraries (scikit-learn, pandas, NumPy): Used by the background ML engine to process activity logs, calculate engagement scores, and generate risk flags.

- iii. Email Delivery Service (SMTP / SendGrid API): Used for sending account verification emails, password reset links and supervisor comment notifications to members.
- iv. File Storage: Member report uploads (PDF, DOCX) will be stored in a structured file storage system (local filesystem or cloud object storage such as AWS S3).
- v. Web Framework: The application will be developed using a Python-based web framework (Django or Flask) with a RESTful API architecture. The frontend will be built using HTML5, CSS3, and a JavaScript framework (e.g., React or Vue.js).

3.4 Communications Interfaces

- i. The system shall send an automatic email verification link to a new user's registered email address during account setup.
- ii. The system shall send an email notification to an member when a supervisor posts a comment on one of their submitted reports.
- iii. The system shall send automated email alerts to supervisors when the ML engine flags an member as disengaged or at risk.
- iv. All data communication between the client and server shall be encrypted using HTTPS (TLS 1.2 or higher).
- v. The system shall use RESTful API communication between the frontend client and the backend server, with all responses formatted in JSON.

4. System Features

4.1 Task Management

This module covers the complete task lifecycle: a supervisor creates, schedules, and assigns a task; the member updates its status and submits progress with supporting documents; and the supervisor reviews the submission and provides feedback. It replaces manual, spreadsheet-based tracking with a single connected workflow where every stage triggers an automatic notification to the relevant party

4.1.1 Create, Track, and Review Tasks

4.1.1.1 Description and Priority

This feature allows a supervisor to create a task — defining its title, description, schedule (deadline), and priority — and assign it to one or more members. The member then updates the

task's status as work progresses and submits proof of progress at completion. Once submitted, the supervisor reviews the report and posts feedback, which the member is notified of and can read. This is a **high priority** feature, as it is the foundational workflow of the entire system

4.1.1.2 Stimulus/Response Sequences

Table 1: Use Case UC1 — Task Management: Stimulus/Response Sequence

Field	Details
Use Case Name	Task Management
Use Case ID	UC1
Priority	High
Actor(s)	Supervisor, member, System
Description	A supervisor creates, schedules, and assigns a task to one or more members. The member updates task status and submits progress with supporting documents. The supervisor reviews the submission and posts feedback, which the member receives.
Preconditions	The supervisor is logged into the system. At least one member exists in the system.
Normal Flow of Events	1. The supervisor navigates to the Task Management panel and clicks "Create New Task." 2. The supervisor enters the task title and description. 3. The supervisor sets the task schedule, including the start date and deadline. 4. The supervisor sets the task priority (High, Medium, or Low). 5. The supervisor selects one or more members from the member list to allocate the task to. 6. The supervisor clicks "Assign Task." 7. The system validates that all required fields are complete.

Field	Details
	8. The system saves the task and links it to each selected member. 9. The system sends an in-app notification to each assigned member, confirming the new task and its schedule. 10. The task appears in each assigned member's task list, showing its title, deadline, and priority. 11. The member opens the task to review its details. 12. As work begins, the member updates the task status to "In Progress." 13. The system records a timestamp for the status change and reflects it on the supervisor's dashboard in real time. 14. As the member continues working, they may update the status again to reflect current progress. 15. Once work is complete, the member opens the submission panel for that task. 16. The member types a progress report directly into the in-app text editor, an uploaded file (PDF or DOCX), or both, as proof of work. 17. The member clicks "Submit." 18. The system validates the submission, confirming the report is not empty and any attached file is a supported format. 19. The system saves the submission, marks the task status as "Completed" or "Under Review," and timestamps the record. 20. The system sends an in-app notification to the supervisor, indicating that a report has been submitted for that task. 21. The submission is added to the member's submission history, visible to both the member and supervisor. 22. The supervisor opens the Review panel and sees a list of recent submissions, showing member name, submission date, and report type. 23. The supervisor selects the relevant submission to open it.

Field	Details
	24. The system renders the report content in the review interface, or displays the attached file in an embedded viewer with a download option. 25. The supervisor reads the submitted report. 26. The supervisor types written feedback into the comment panel. 27. The supervisor clicks "Post Comment." 28. The system saves the feedback, associating it with the report, the supervisor's identity, and a timestamp. 29. The system sends an in-app notification to the member, indicating new feedback has been posted. 30. The member opens the relevant report and reads the supervisor's feedback. 31. All comments on the report are displayed in chronological order, each showing the commenter's name and timestamp.
Post Conditions	The task, all status updates, the submitted report, and the supervisor's feedback are saved in the database and are visible to both the supervisor and the relevant member(s) at every stage.
Alternative Flows	Flow 1 — Incomplete Task Form: If the supervisor attempts to assign a task with the title or deadline missing, the system displays an inline validation error and does not save the task; the supervisor must correct the form before proceeding. Flow 2 — Invalid Submission: If the member submits an empty report or attaches an unsupported file type, the system displays a descriptive validation error and does not save the submission; the member must correct and resubmit. Flow 3 — Empty Feedback: If the supervisor attempts to post an empty comment, the system displays a validation warning and does not save it.

Field	Details
	Flow 4 — Multiple Reviewers: If more than one supervisor has access to the same member's reports, the system supports multiple comments from different supervisors on the same submission, all displayed in chronological order. Flow 5 — Editing Feedback: A supervisor may return to a previously posted comment to edit or delete it; the system records the edit timestamp and updates the displayed comment accordingly.

SRS For Create, Track, and Review Tasks

Table 2: SRS Detail — Task Management

Category	Requirement	Input	Output	Business Rule
Creation	The system shall allow a supervisor to create a new task with a title, description, deadline, and priority level	Task title, description, deadline (date), priority level (e.g. High/Medium/Low)	Saved task record in the database	Title and deadline are mandatory fields; task cannot be saved without them
Assignment	The system shall allow a supervisor to allocate a task to one or more members	Selected member(s) from existing user list	Task linked to selected member(s) in the database	At least one member must exist in the system before assignment can occur
Notification	The system shall notify each assigned member via in-app notification upon task assignment	Completed task assignment action	In-app notification delivered to each assigned member	Notification must be triggered immediately upon task save

Status Update	The system shall allow an member to update the status of an assigned task	Selected task; new status value (To-Do, In Progress, Completed)	Updated status saved with a timestamp; supervisor dashboard reflects change in real time	Only the member(s) assigned to the task may update its status
Submission	The system shall allow an member to submit proof of progress as a typed report and/or an uploaded file	Typed report text and/or uploaded file (PDF or DOCX)	Submission saved, linked to the task and member, timestamped	Empty submissions and unsupported file types are rejected with a descriptive error
Submission Notification	The system shall notify the supervisor when an member submits a status update or report	Completed submission action	In-app notification delivered to the supervisor	Notification must trigger immediately upon successful submission
Submission History	The system shall maintain a full submission history for each member	All past submissions linked to the member	Chronological submission history visible to the member and their supervisor	Both member and supervisor must have read access to the full history
Review	The system shall allow a supervisor to open and read any submitted report in full within the platform	Supervisor selection of a report from the submission list	Full report content rendered in the review interface, or embedded viewer/download link for file submissions	Only reports from the supervisor's own members are accessible
Feedback	The system shall allow a supervisor to post written	Typed feedback text; report ID; supervisor session	Feedback saved and associated with the report, supervisor	Supervisor must have the report open in the review interface

	feedback on a submitted report		identity, and timestamp	before posting feedback
Feedback Notification	The system shall notify the member when new feedback is posted on their report	New feedback saved event	In-app notification delivered to the member	Notification must trigger immediately upon feedback save
Feedback Display	The system shall display all feedback on a report in chronological order	All feedback entries associated with a report	Ordered feedback thread showing commenter name, timestamp, and comment text	Oldest feedback appears first
Edit/Delete Feedback	The system shall allow a supervisor to edit or delete their own previously posted feedback	Edit or delete action on existing feedback; supervisor session	Updated feedback with edit timestamp, or feedback removed	Supervisors may only edit or delete feedback they authored

4.2 Performance Monitoring

4.2.1 Description and Priority

This feature provides a supervisor with an auto-generated analytics dashboard showing aggregated performance metrics for all their members. The dashboard computes an overall Performance Index (PI) by evaluating five core behavioral and technical metrics: Timeliness, Quality, Peer Review, Responsiveness, and Collaboration. These five metrics are consolidated into the three weighted components of the PI formula: Timeliness drives Task Performance (TP); Peer Review and Collaboration drive Peer Evaluation (PE); and Quality and Responsiveness drive Supervisor Assessment (SA). This is a high priority feature.

4.2.1.1 Mathematical Breakdown of the Metrics Framework

The overall Performance Index (PI) is calculated using a weighted linear combination:

$$PI = w_1 \cdot TP + w_2 \cdot PE + w_3 \cdot SA$$

Business Constraint: The weights are system-configurable constants satisfying the normalization condition: $w_1 + w_2 + w_3 = 1.0$. The system applies the following baseline distribution to prioritize quality over speed:

- $w_1 = 0.25$ (Task Performance driven by Timeliness)
- $w_2 = 0.25$ (Peer Evaluation splitting program-wide review and team collaboration)
- $w_3 = 0.50$ (Supervisor Assessment focused on Output Quality)

1. Task Performance (TP): Fueled by Timeliness

Timeliness: Each assigned task receives a tiered score (0–5) based on how the final submission timestamp compares to the deadline.

- On or before the deadline = 5/5
- Late by less than 24 hours = 3/5
- Late by 24 hours to 3 days = 2/5
- Late by more than 3 days = 1/5
- Never submitted, deadline passed and task still open = 0/5

$$\text{Timeliness} = (\text{Sum of Tier Scores Across Assigned Tasks} / 5) / \text{Total Tasks Assigned}$$

$$TP = (\text{Tasks Completed} / \text{Total Tasks Assigned}) \times \text{Timeliness}$$

2. Peer Evaluation (PE): Fueled by Peer Review & Collaboration

Peer Review & Collaboration: Peer-to-peer ratings mapping workload contribution and group interaction team scores.

Metrics Breakdown:

- Peer Review (PR): Quantitative score (1–5) assigned by any program member from the wider member cohort, evaluating the member's visible technical contributions, presentations, or shared engineering artifacts.
- Collaboration (CO): Quantitative score (1–5) assigned exclusively by immediate project group teammates, evaluating the member's daily communication consistency, reliability, and helpfulness within their core team.

$$PE = ([\text{Sum of PR} / (5 * k)] * 0.5) + ([\text{Sum of CO} / (5 * m)] * 0.5) - P$$

Where k is the dynamic number of program cohort members who evaluated the member's public work, and m is the number of immediate project teammates who rated their core team collaboration.

P is the active penalty deduction (0.10) systematically applied if the member fails to submit their own mandatory peer feedback forms before the system window closes.

Assessment Anonymity: Peer Review and Collaboration submissions enforce one-directional anonymity. The supervisor retains full visibility of which member submitted each assessment and the individual scores given. The member being assessed, however, views only their aggregated PR and CO scores together with anonymized written comments, and can never see which peer submitted a given rating or comment.

Business Rule – Individual Tasks: Peer Evaluation (PE) only applies to tasks assigned to a group; no peer assessment occurs for tasks assigned to a single member. When a task has no group, PE's weight is redistributed between Task Performance (TP) and Supervisor Assessment (SA), preserving their original 0.25:0.50 ratio (1:2). For individually assigned tasks, the Performance Index is calculated as $PI = (1/3) \cdot TP + (2/3) \cdot SA$, equivalent to $TP \approx 33.3\%$ and $SA \approx 66.7\%$.

3. Supervisor Assessment (SA): Fueled by Quality & Responsiveness

Quality & Responsiveness: Captured during the supervisor's weekly report evaluation workflow.

$$SA = (Quality\ Score + Responsiveness\ Score) / Maximum\ Possible\ Score\ (10)$$

Responsiveness Metric Rules: Evaluates the speed of engineering iterations by tracking elapsed hours between a requested revision and a subsequent report resubmission:

- Resubmissions within 24 hours = 5/5
- Resubmissions within 24 to 48 hours = 3/5
- Resubmissions beyond 48 hours = 1/5

4.2.1.2 Stimulus/Response Sequences

Table 3: Use Case UC7 – View Analytics Dashboard: Stimulus/Response Sequence

Field	Details
Use Case Name	View Analytics Dashboard
Use Case ID	UC7
Priority	High
Actor(s)	Supervisor, System
Description	A supervisor views an automatically generated analytics dashboard showing aggregated member performance data, utilizing a quality-biased formula split between program-wide evaluations and team collaboration mechanics.
Preconditions	The supervisor is logged in. Sufficient task, peer feedback data, and report submission records exist in the database.
Normal Flow of Events	1. The supervisor navigates to the Performance Monitoring Analytics section. 2. The system queries the relational database to fetch raw tracking logs (completed tasks, submission timestamps, peer feedback forms, and supervisor quality marks).

Field	Details
	3. The system applies the weighted Performance Index formula ($PI = w_1 \cdot TP + w_2 \cdot PE + w_3 \cdot SA$). 4. The system splits the Peer Evaluation (PE) variable by calculating the average of cohort-wide Peer Reviews (PR) and internal Team Collaboration (CO) metrics independently, while checking for missed evaluation quotas to apply the automatic penalty ($P = 0.10$). 5. The system calculates the Responsiveness sub-metric based on exact turnaround durations (<24h, 24–48h, >48h) between supervisor revision requests and subsequent report re-uploads. 6. The system dynamically aggregates and renders the metrics into distinct, high-fidelity visual UI elements: standalone Task Performance (TP) bar charts, iteration line graphs for Responsiveness trends, and side-by-side matrices separating cohort metrics from local team dynamics. 7. The supervisor interacts with the filtering UI panel to isolate data by a specific member or a specific evaluation cycle date range. 8. The system re-filters the visual dashboard scope instantaneously to reflect only the selected parameters.
Post Conditions	The supervisor has a clear visual summary of both cohort-level and individual member performance.
Alternative Flows	Flow 1: If insufficient data exists (e.g., no tasks have been assigned yet), the system displays a message explaining that analytics will populate as activity accumulates. Flow 2: If a member fails to submit their minimum required evaluation forms for other program cohort members within the evaluation window, the background aggregation service systematically subtracts 0.10 from the total PE score, displays a

Field	Details
	"Penalty Applied" status tag next to their peer metrics card, and locks the final adjusted PI output.

SRS For Performance Monitoring

Table 4: SRS Detail – Performance Monitoring

Category	Requirement	Input	Output	Business Rule
Aggregation	The system shall automatically aggregate task logs, report turnaround metrics, cohort peer reviews, and teammate feedback profiles into structured sub-scores.	Task records, submission timestamps, peer feedback entries, and supervisor quality logs.	Aggregated performance metrics available for dashboard rendering	Aggregation must cover all members under the supervisor's supervision
Visualization	The system shall render an analytical interface containing metric breakdown charts for the calculated index values.	Aggregated performance metrics	Dashboard with bar charts (TP and Timeliness), line graphs (Responsiveness trends), and grids (Peer Review and Collaboration).	All three chart types are mandatory for the dashboard to be considered complete
Filtering	The system shall allow the supervisor to filter analytics views by individual member and date range	Filter parameters (member selection; date range)	Filtered analytics view scoped to selected member or period	Both filter types must be independently operable

Refresh	The system shall refresh analytics data at regular intervals or on demand	Scheduled timer (minimum every 24 hours) or manual refresh request from supervisor	Updated analytics data rendered on the dashboard	Automatic refresh must occur at least once every 24 hours; supervisor may also trigger a manual refresh at any time
Penalty Logic	The system shall enforce a performance index penalty (P) for members who do not submit peer evaluations.	Completed peer review forms and active evaluation logs.	Adjusted Peer Evaluation (PE) matrix score values.	If a member fails to submit peer reviews for their assigned group members by the closing window, penalty P (0.10) is systematically deducted.
Timeliness Scoring	The system shall score each assigned task's timeliness using a tiered scale rather than a binary on-time/late check.	Task deadline timestamp and final submission timestamp (or absence of submission).	Per-task Timeliness tier score (0–5), feeding the averaged Timeliness value used in the TP formula.	On time = 5/5; late under 24h = 3/5; late 24h–3 days = 2/5; late over 3 days = 1/5; never submitted and overdue = 0/5.
Individual Task Reweighting	The system shall redistribute the Peer Evaluation weight between Task Performance and Supervisor Assessment for tasks with no assigned group.	Task assignment type (individual or group).	Reweighted Performance Index for the task, excluding PE.	For individually assigned tasks, $PI = (1/3) \cdot TP + (2/3) \cdot SA$, preserving the underlying 0.25:0.50 weight ratio.
Responsiveness	The system shall measure the elapsed hours between a requested revision	Revision workflow timestamps and	Calculated quantitative Responsiveness sub-metric.	- Resubmissions within 24 hours = 5/5 24 to 48 hours = 3/5

	and a subsequent report resubmission.	resubmission events.		• Beyond 48 hours = 1/5.
Assessment Visibility	The system shall enforce one-directional anonymity between assessors and assessees for peer assessments.	Submitted peer assessments with assessor and assessee identities, Likert scores, and comments.	Supervisor view: full assessor-to-assessee detail. Assessee view: aggregated scores plus anonymized comments only.	An assessee must never see which peer submitted a given rating or comment; the supervisor must always see full attribution.
Empty State	If insufficient data exists, the system shall display an explanatory message	No task or submission data found	Message informing the supervisor that analytics will populate as activity accumulates	No charts or metrics should be displayed when data is insufficient; message must be shown instead

4.3 Machine Learning Engagement Monitoring

4.3.1 Generate Engagement Score and Risk Alerts

4.3.1.1 Description and Priority

This feature describes the background ML engine that monitors member behavioral patterns — including login frequency, task update frequency, and report submission timeliness — and calculates a continuous Engagement Score for each member. When a score drops below a configurable threshold, the system flags the member as "at risk" and adds them to the supervisor's alert list. This is a **high priority** feature.

4.3.1.2 Stimulus/Response Sequences

Table 5: Use Case UC8 – Generate Engagement Score and Risk Alerts: Stimulus/Response Sequence

Field	Details
Use Case Name	Generate Engagement Score and Risk Alerts

Field	Details
Use Case ID	UC8
Priority	High
Actor(s)	System (ML Engine), Supervisor
Description	The ML engine analyzes member activity data, calculates engagement scores, and surfaces at-risk flags on the supervisor's dashboard.
Preconditions	At least two weeks of member activity data (logins, task updates, report submissions) have been collected.
Normal Flow of Events	1. The ML engine runs as a scheduled background job (e.g., nightly). 2. It reads activity logs from the database for each member. 3. It calculates an Engagement Score (0–100) based on weighted behavioral features including login frequency, task update recency, and submission timeliness. 4. Members scoring below the configured risk threshold (default: 40) are flagged as "at risk." 5. The supervisor's dashboard displays updated engagement scores and the current at-risk alert list. 6. An email alert is sent to the supervisor for each newly flagged member.
Post Conditions	Engagement scores are updated in the database. At-risk members surfaced prominently on the supervisor's dashboard.
Alternative Flows	Flow 1: If a member has insufficient activity history (less than two weeks of data), the system displays "Insufficient data" instead of a score and does not generate a risk flag for that member.

SRS For Machine Learning Engagement Monitoring

Table 6: SRS Detail – Machine Learning Engagement Monitoring

Category	Requirement	Input	Output	Business Rule
Score Calculation	The system shall include a background ML process that calculates an Engagement Score (0–100) for each member	Login frequency logs, task update frequency logs, report submission timeliness records	Engagement Score (0–100) per member stored in the database	Score must be calculated using weighted behavioral features; exact weighting is configurable
Risk Flagging	The system shall flag members whose Engagement Score falls below a configurable threshold as "at risk"	Calculated Engagement Score; configured risk threshold (default: 40)	"At risk" flag applied to the member's profile	Default threshold is 40; threshold must be configurable by the supervisor or system admin
Dashboard Display	The system shall display current engagement scores and at-risk flags on the supervisor's analytics dashboard	Stored engagement scores and at-risk flags	Engagement scores and prominently surfaced at-risk alerts on the supervisor's dashboard	Scores and flags must be visible without additional navigation
Email Notification	The system shall send an email notification to the supervisor when a member is newly flagged as at risk	Newly generated "at risk" flag for a member	Email notification sent to the assigned supervisor	Notification is sent only when a member is <i>newly</i> flagged, not on every score recalculation
Isolation	The ML engine shall run as an isolated background process and shall not degrade the performance of the live web application	Scheduled background job trigger (e.g. nightly)	Updated scores and flags written to the database	The ML engine must be decoupled from the web application process

Insufficient Data	The system shall display "Insufficient data" for members with less than two weeks of activity history, without generating a risk flag	Member activity log with less than two weeks of data	"Insufficient data" label displayed instead of a score; no risk flag generate	Two weeks of activity data is the minimum required to calculate a score or generate a flag
-------------------	---	--	---	--

5. Non-Functional Requirements

5.1 Performance Requirements

- i. The system shall load any dashboard page within 3 seconds under normal network conditions (10 Mbps or higher) for up to 100 concurrent users.
- ii. Report submission and task status updates shall be processed and confirmed to the user within 2 seconds.
- iii. The ML background engine shall complete its scheduled engagement score recalculation within 10 minutes for a dataset of up to 500 members.

5.2 Security Requirements

- i. All data in transit between the client and server shall be encrypted using TLS 1.2 or higher (HTTPS).
- ii. All user passwords shall be stored as salted cryptographic hashes (e.g., bcrypt). Plaintext passwords shall never be stored or logged.
- iii. The system shall implement role-based access control (RBAC) enforced at the API level, preventing any user from accessing data outside their authorized scope.
- iv. The system shall log all login attempts — including failed attempts — with timestamps and IP addresses.
- v. User sessions shall expire after 30 minutes of inactivity, requiring re-authentication.

5.3 Reliability and Availability

- i. The system shall target an uptime of 99.5% or higher during business hours. The system shall perform automated database backups at least once every 24 hours, with backup data retained for a minimum of 30 days.

- ii. In the event of an ML engine failure, the main web application shall continue to function normally without any degradation to task management, reporting, or commenting features.

5.4 Usability Requirements

- i. A new member user shall be able to submit their first report and update a task status within 10 minutes of first login, without requiring external guidance.
- ii. All UI interfaces shall be fully responsive and usable on screen widths as narrow as 375px (standard smartphone viewport).
- iii. The system shall follow WCAG 2.1 Level AA accessibility guidelines for color contrast, keyboard navigation, and screen reader compatibility.

5.5 Maintainability

- i. The codebase shall follow a modular architecture, clearly separating the web application layer, database layer, and ML engine into distinct, independently deployable components.
- ii. All API endpoints shall be documented using an OpenAPI (Swagger) specification and kept up-to-date with each release.

5.6 Data Privacy and Compliance

- i. The system shall comply fully with Uganda's Data Protection and Privacy Act (2019), including implementing mechanisms for data access requests and data deletion upon user departure or request.
- ii. Personal data (names, email addresses, phone numbers, performance records) shall not be shared with any third-party service without explicit consent, except where legally required.

Entity-Relationship Diagram

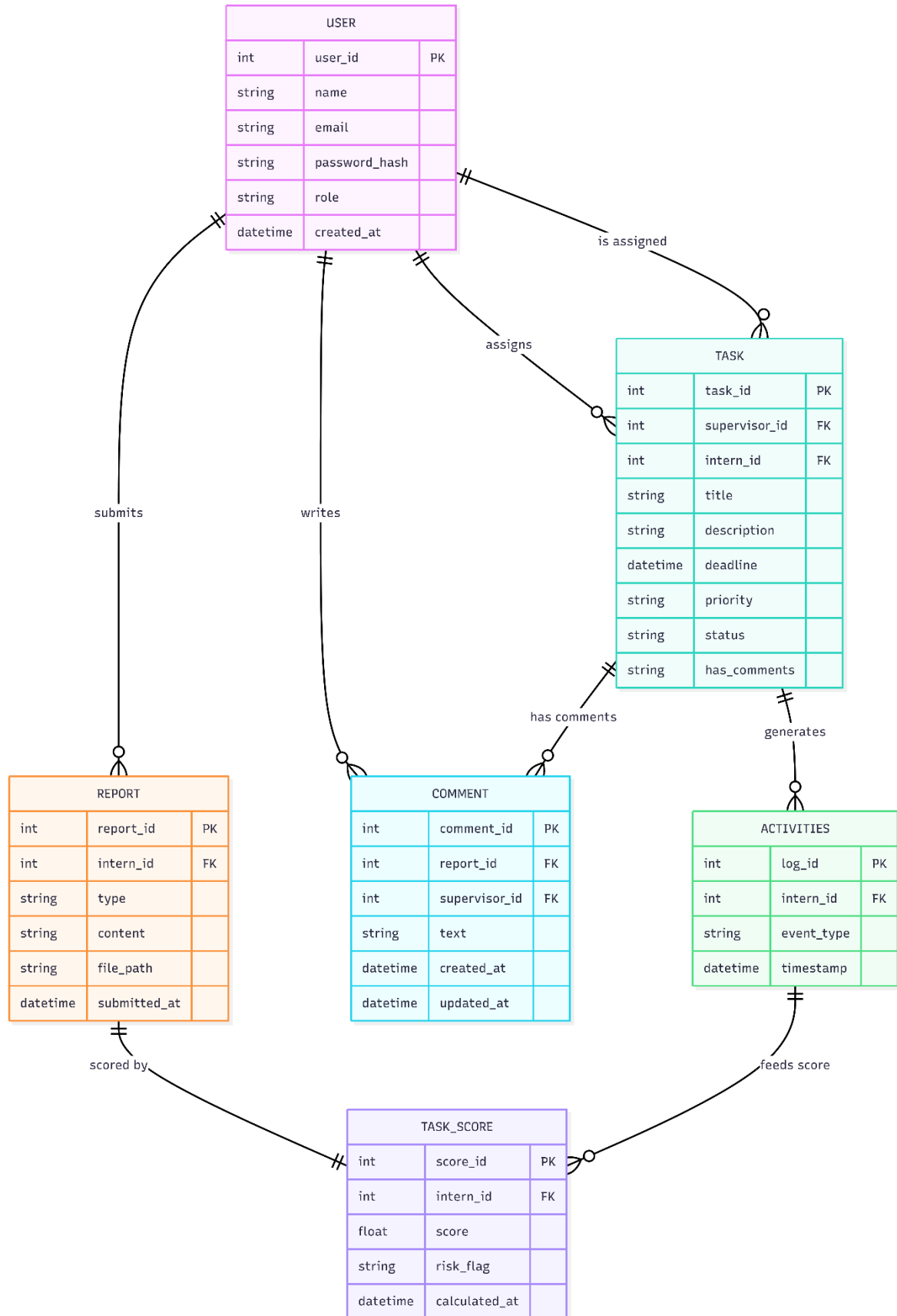


Figure 3: Entity-Relationship Diagram – Task Management System

6. References

1. Uganda Data Protection and Privacy Act, 2019.
2. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
3. Scikit-learn Documentation — <https://scikit-learn.org/stable/documentation.html>
4. Django REST Framework Documentation — <https://www.django-rest-framework.org/>
5. Concept paper for the Member Management System (internal document).

Appendix A: Glossary

Table 7: Glossary of Terms

Term	Definition
API	Application Programming Interface, a set of protocols that allow software systems to communicate.
CRUD	Create, Read, Update, Delete, the four basic operations performed on data records.
Engagement Score	A numerical value (0–100) calculated by the ML engine to represent how actively an member is engaging with the system based on their behavioral patterns.
GB	Gigabyte
HTTPS	Hypertext Transfer Protocol Secure — the encrypted web communication protocol.
Member	A user of the system who is undergoing a supervised internship program within the organization.

Term	Definition
JSON	JavaScript Object Notation — a lightweight data interchange format used in API responses.
Kanban	A visual workflow management method displaying tasks organized in columns by status.
ML	Machine Learning — a method of data analysis that automates building analytical models from patterns in data.
MySQL	My Structured Query Language — an open-source relational database management system.
ORM	Object-Relational Mapper — a tool that converts database records into programming language objects.
RBAC	Role-Based Access Control — a method of restricting system access based on the roles assigned to individual users.
REQ	Requirement
REST	Representational State Transfer — an architectural style for building web APIs.
SRS	Software Requirements Specification

Term	Definition
Supervisor	A user of the system responsible for managing, assigning tasks to, and reviewing the work of members.
TLS	Transport Layer Security — a cryptographic protocol for securing network communications.
UC	Use Case
WCAG	Web Content Accessibility Guidelines — international standards for making web content accessible.

Appendix B: Use Case Summary

The following use cases are defined in this document and should be represented in the system use case diagram:

Table 8: Use Case Summary

Use Case ID	Use Case Name	Primary Actor	Priority
UC1	Create and Assign Task	Supervisor	High
UC2	Update Task Status	Member	High
UC3	View Member Progress	Supervisor	High
UC4	Submit Report	Member	High

Use Case ID	Use Case Name	Primary Actor	Priority
UC5	Review Submitted Report	Supervisor	High
UC6	Add Comment to Member Report	Supervisor	High
UC7	View Analytics Dashboard	Supervisor	High
UC8	Generate Engagement Score and Risk Alerts	System (ML Engine)	High

Actor Descriptions:

- i. Member: Interacts with UC2 and UC4. Can view their own task list, notifications, and report feedback comments.
- ii. Supervisor: Interacts with UC1, UC3, UC5, UC6, and UC7. Receives dashboard outputs from UC8.
- iii. System (ML Engine): Automatically triggers UC8 on a scheduled basis; outputs are surfaced to the Supervisor via the dashboard and email alerts.
- iv. System Administrator: Manages user accounts, role assignments, and system configuration. Operates outside the functional UCs above but is a recognized system actor.